UQAC

Université du Québec
à Chicoutimi

Atelier 1 : Mobile Forensics

8INF968 - Atelier pratique en cybersécurité I

Fehmi Jaafar Samuel Desbiens

Justin Bossard Mattéo Gouhier Paul Mathé Samuel Plet

3 octobre 2025

Table des matières

Introduction **3**

1 *Identification* et *Collection* **4**

2 *Examination* **5**
2.1 Mise en place de l'environnement de travail 5
2.1.1 Installation d'Autopsy 5
2.1.2 Création du projet 5
2.1.3 Ajout des données 5
2.1.4 Exploitation des données 5
2.2 Identité 6
2.3 Photographies 6
2.3.1 Transactions Bitcoin 7
2.4 Localisation 7
2.5 Contacts 8
2.6 Messages 9
2.6.1 Examen du code source d'OmniNotes 9
2.6.2 Messages décodés 11
2.7 Skimming 12

3 *Analysis* **13**

Conclusion **14**

Table des figures

2.1 Informations civiles - /system_ce/0/snapshots/141_reduced.jpg 6
2.2 Selfie - /media/0/DCIM/Camera/IMG_20210411_144226.jpg 6
2.3 *Piadina* - /media/0/DCIM/Camera/20210420_125202.jpg 7
2.4 Adresse Bitcoin - /clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg 7
2.5 Carte des photos géolocalisées 8
2.6 Billet d'avion - /clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg 8

Introduction

L'objectif de ce premier projet de *Mobile Forensic* est de réaliser une investigation sur une image de smartphone, dans un contexte préétabli, afin de mettre en évidence des preuves numériques pertinentes pour une enquête.

Par définition, la *Digital Forensics*, ici appliqué aux environnements mobiles, regroupe *l'ensemble de méthodes et d'outils visant à récupérer, préserver et examiner des preuves numériques issues ou transmises par des dispositifs électroniques.*
Ce domaine s'appuie sur le principe de Locard[1], et vise à répondre aux 5WH[2].

L'investigation doit préserver l'intégrité des preuves et garantir une acquisition recevable juridiquement, en maintenant une chaîne de confiance.

Le processus légal d'investigation est souvent le suivant :

1. **Identification** Détection de l'incident, reconnaissance et sécurisation des sources de preuves.
2. **Collection** Acquisition des données de l'appareil avec des méthodes garantissant l'intégrité (copies, empreintes numériques).
3. **Examination** Extraction, tri et préparation des données pertinentes (fichiers, métadonnées, artefacts cachés).
4. **Analysis** Interprétation des preuves pour établir les faits, reconstituer les événements et relier les acteurs.
5. **Presentation** Restitution structurée des résultats sous forme de rapport recevable juridiquement.

Pour ce travail, les étapes d'*Identification* et de *Collection* ont déjà été effectués. Nous les présenterons dans un premier temps, avant de présenter successivement nos résultats pour *Examination* et *Analysis*. Ce rapport, dans son ensemble, constitue la phase de *Presentation*.

[1] Chaque interaction laisse une trace
[2] Who, what, when, where, why, and how

Partie 1

Identification et *Collection*

> L'enquête présentée dans ce rapport s'inscrit dans une progression en 5 étapes du Challenge 2021 de la DFRWS, nous en sommes à la troisième étape.

Le 9 avril 2021, un dispositif de skimming a été découvert sur un distributeur automatique de la Poste suisse, à Lausanne. Une micro-SD intégrée au dispositif a été saisie et a révélé des traces d'utilisation frauduleuse liées à des cartes bancaires.

À partir de ces informations, une perquisition a été menée le 18 avril 2021 à Aoste, Italie, où un laboratoire de fabrication a été découvert. Un Raspberry Pi connecté à une imprimante 3D a pu être saisi. Une copie de sa micro-SD a permis d'étudier son rôle éventuel dans la production d'objets illicites.

L'exploitation de ces premiers résultats a conduit à l'identification d'un suspect, interpellé le 20 avril 2021 à l'aéroport de Genève alors qu'il tentait de voyager avec un billet acheté à l'aide d'une carte bancaire volée. Son téléphone a été saisi et une image complète du système de fichiers a été extraite le lendemain. Une analyse préliminaire a révélé la présence de SMS encodés, susceptibles de contenir des éléments de preuve pertinents pour la compréhension des faits.

Notre investigation se basera donc sur l'image en question.

- https://cfreds.nist.gov/all/FrancescoServida%2FDFRWS/SamsungGalaxyS10Android10DFRWS2021
- SHA2-256 : `54877505f1b4eb26c4cb6b43fd6338424660c207e678b773044a4a79d6e374b7`

Partie 2

Examination

La racine des figures correspond à `Dump/data/`

Pour examiner l'image, nous allons utiliser principalement le logiciel Autopsy. Celui-ci permet d'analyser et d'extraire toutes les informations potentiellement utiles d'une image disque : métadonnées, conversations, journal d'appels, mails, contacts…

Nous allons commencer par montrer le plus visible, en allant vers les données plus compliquées à extraire, ce qui inclus les messages encodés, mentionnés précédemment comme étant pertinents.

2.1 Mise en place de l'environnement de travail

2.1.1 Installation d'Autopsy

Pour installer Autopsy, il suffit de se rendre sur la page de téléchargement. On peut l'installer sur GNU/Linux, MacOS et Windows, les instructions pour chaque OS sont détaillées sur le site officiel.

2.1.2 Création du projet

Après installation, on peut créer le projet. On démarre Autopsy, on sélectionne `New Case`, puis on nomme le projet `Samsung`, par exemple.
On remplit les informations optionnelles de manière adéquate, puis on finit la création du projet.

2.1.3 Ajout des données

Après création du projet, on peut ajouter les données. Dans l'ordre :

1. `Generate new host name based on data source name`
2. `Logical files`
3. `Local files and folders`, puis on ajoute le `zip` directement.
4. On laisse tout coché, sauf `iOS analyzer` (et `Android Analyzer (aLEAPP)` si on n'a pas Windows)
5. On termine, et on attend patiemment la fin de l'analyse

2.1.4 Exploitation des données

TODO

2.2 Identité

Commençons par présenter le propriétaire du téléphone : il se nomme Werner von Strucker[1].

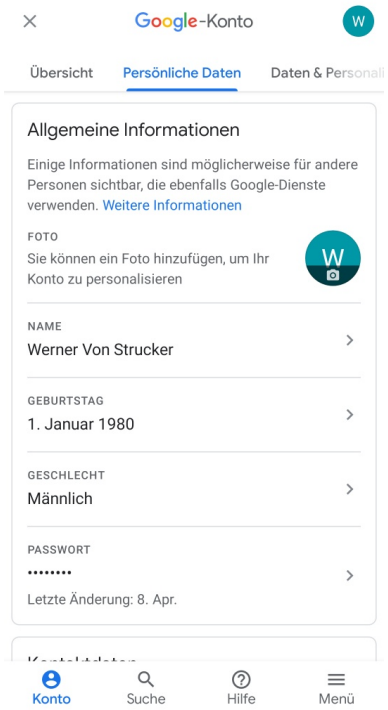


Figure 2.1: Informations civiles - `/system_ce/0/snapshots/141_reduced.jpg`

2.3 Photographies

Dans les photos, un certain nombre semble intéressant.

Tout d'abord, un selfie de Werner.

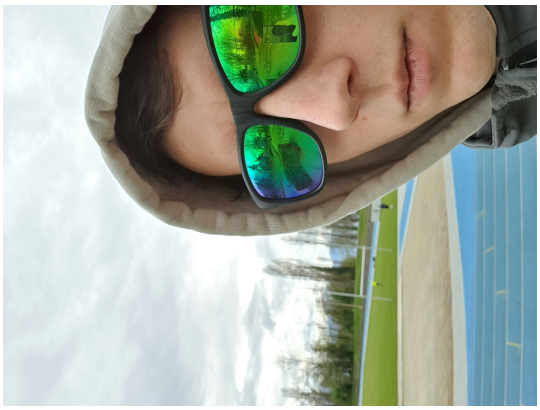


Figure 2.2: Selfie - `/media/0/DCIM/Camera/IMG_20210411_144226.jpg`

Par ailleurs, une photo d'une *piadina*, scène triviale d'alimentation, illustre qu'il emploie son téléphone à titre personnel. D'autres photos vont dans ce sens.

[1] Il s'agit du nom d'un caractère fictif de Marvel, lié à HYDRA. Voir https://en.wikipedia.org/wiki/Werner_von_Strucker



Figure 2.3: *Piadina* - `/media/0/DCIM/Camera/20210420_125202.jpg`

2.3.1 Transactions Bitcoin

Une capture d'écran avec une adresse Bitcoin est présente.



Figure 2.4: Adresse Bitcoin - `/clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg`

En regardant sur la blockchain, on retrouve l'historique de transactions de cette adresse : https://www.blockchain.com/explorer/addresses/bch/qpuv475w4wytd6agujpzfl9muzajf9le45nkefd23y.

On identifie les adresses BTC avec leurs 4 premiers et leurs 4 derniers caractères. On note une transaction de la même manière.

L'adresse en question a deux transactions associées, impliquant 5 autres adresse BC :

- d14c-d8f2
- bddf-e925

2.4 Localisation

16 photos ont une localisation associée. On voit donc que l'utilisateur en a pris uniquement en Suisse, principalement aux abords de Lausanne.

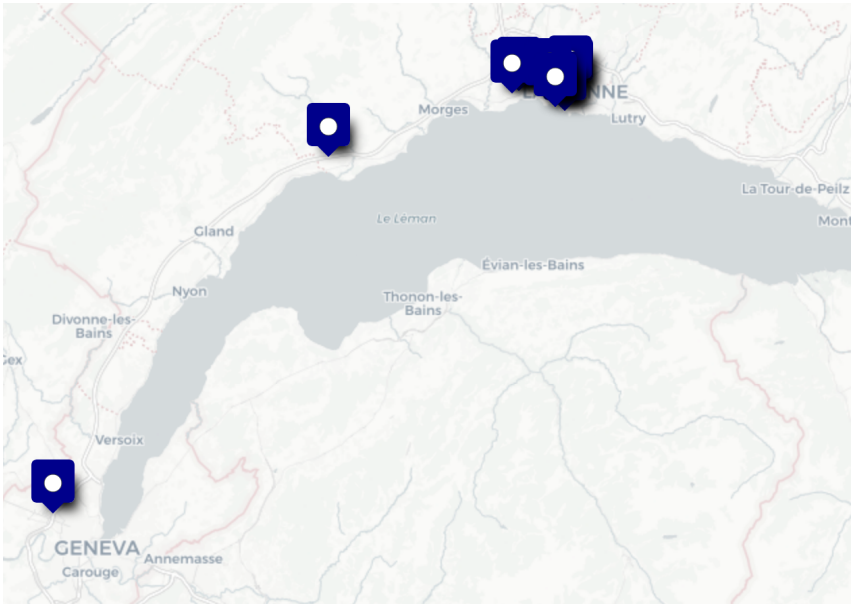


Figure 2.5: Carte des photos géolocalisées

Une capture d'écran avec un vol, prévu à la date d'arrestation de Werner, est également accessible. Elle date de 18 avril, montrant que le vol a été pris relativement tard.

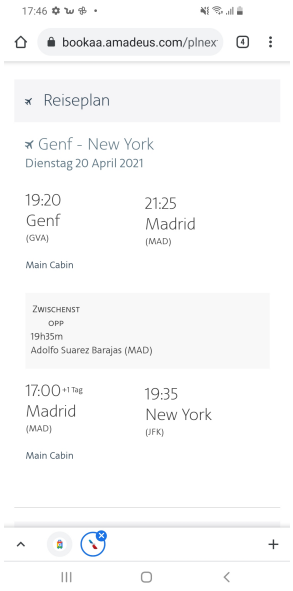


Figure 2.6: Billet d'avion - `/clipboard/1740051371957_356_87/Screenshot_20210418-161954_BRD.jpg_thum.jpg`

2.5 Contacts

Une base de données GMail[2] donne un aperçu de ses contacts. Notons l'adresse `therealredskull41@gmail.com`. En effet, Red Skull est un personnage fictif du même univers que Werner von Strucker.

[2] `/data/com.google.android.gm/databases/peopleCache_vonstruckerwerner@gmail.com_com.google_11.db`

2.6 Messages

L'examen des données extraites du téléphones du suspect a permis de révéler la présence de messages illisibles reçus et envoyés, suggérant l'utilisation de méthodes d'encryption des messages.

Par exemple : `2 2 +41786909109 1617901761280 0 1 -1 2 HHY6cKEMFlNU3NPs/bxhxPKQ RyDXUrQK7a8tHtX5xBXdYQ= 0 0 1 it.feio.android.omninotes 1 0 0 228015930119996 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0`

L'analyse des métadonnées associées à ces messages met en évidence la provenance de ces messages, OmniNotes, une application open-source de prise de notes. Voir des messages cryptés depuis une application de prise de notes a priori non dédiée à l'envoi de message est un indicateur fort de tentative de dissimulation.

La prochaine étape est d'analyser le code source de cette application présent sur l'appareil pour identifier les méthodes utilisées.

2.6.1 Examen du code source d'OmniNotes

Pour extraire le code de l'application présente sur l'appareil, nous avons utilisé apk2java-linux.

L'examen des fonctions a confirmé son rôle dans l'envoi de messages, notamment via l'appel d'une fonction dédiée à l'envoi.

```
1  public void putComm(View object) {
2      object = (EditText)this.findViewById(2131296976);
3          object = new
   ↪   TextHelper(this.getResources()).put(object.getText().toString());
4          if (object.length() > 160) {
5              Toast.makeText((Context)this,
   ↪   (CharSequence)ZipHelper.decomp("eJwFQLENwDAIe8UH5I1u6ZL2gQgs1AUPqEuuR1eanI
   ↪   6bVTuIV8JUxsCi/0Y832ED99UM+Q==", (int)0).show();
6              return;
7          }
8          if (object.equals((Object)"")) {
9              Toast.makeText((Context)this,
   ↪   (CharSequence)ZipHelper.decomp("eJwFgLEJACAMBFfJMpaCK6R4QhqV+/mLsOARB/2kb8
   ↪   WWnaUBZ9oIuA==", (int)0).show();
10             return;
11         }
12         StringBuilder stringBuilder = new StringBuilder();
13         stringBuilder.append(ZipHelper.decomp("eJwFQLEJADAMesU7+kOmXlCoSBYX7x
   ↪   /kOn8tDJMnHhQ48AYI"));
14         stringBuilder.append((String)object);
15         stringBuilder.append(" to ");
16         stringBuilder.append(this.rsn());
17         Toast.makeText((Context)this, (CharSequence)stringBuilder.toString(),
   ↪   (int)0).show();
18         PendingIntent.getActivity((Context)this, (int)0, (Intent)new
   ↪   Intent((Context)this, CommActivity.class), (int)0);
19         SmsManager.getDefault().sendTextMessage(this.rsn(), null,
   ↪   (String)object, null, null);
```

```
}
```

L'analyse a mis en évidence l'utilisation de blocs de données encodés :

```
private String get_data() throws IOException, JSONException {
        String string2 =
         ↪  ZipHelper.decomp("eJwNyUsKwDAIBNATZSTbHietkH5wShUkt7frNyNe30Tm6hg
         ↪  PB3Z+ijztYDpM45eW3rrcunA5rQDMDhIV");
        Object object = new StringBuilder();
        object.append(string2);
        object.append(ZipHelper.decomp("eJwFwIEIAAAAACC0/a1OAEAAQA=="));

 ↪  object.append(ZipHelper.decomp(this.resources.getString(2131821101)));
        string2 = (HttpsURLConnection)new
 ↪  URL(object.toString()).openConnection();
        try {
            String string3;
            object = new BufferedReader((Reader)new
 ↪  InputStreamReader(string2.getInputStream()));
            Object object2 = new StringBuffer();
            while ((string3 = object.readLine()) != null) {
                object2.append(string3);
            }
            object = new
 ↪  JSONObject(String.valueOf((Object)object2)).getString("key");
            object2 = new
 ↪  SimpleDateFormat(ZipHelper.decomp("eJwNwIEAAAAMhECiULJ4idm3u4eyBRWSA6E="),
 ↪  Locale.getDefault()).format(new Date());
            string3 = new StringBuilder();
            string3.append((String)object);
            string3.append((String)object2);
            object = string3.toString();
            return object;
        }
        finally {
            string2.disconnect();
        }
}
```

Avec des bouts de code codés en `base64` et compressé en zlib, identifiable grâce au préfixe `eJw` :

- `eJwNyUsKwDAIBNATZSTbHietkH5wShUkt7frNyNe30Tm6hgPB3Z+ijztYDpM45eW3rrcunA5rQDMDhIV` nous donne le lien vers un site nous permettant de récupérer une clé : `rb5CuefkDLyb9T`
- `eJwNwIEAAAAMhECiULJ4idm3u4eyBRWSA6E` nous donne `yyyy-MM-dd`.

En analysant ce bout de code on comprend que l'on va récupérer la clé sur le site Web, on récupère aussi la date actuelle que l'on va mettre sous format `yyyy-MM-dd` et que l'on va concaténer à la clé. Ce qui va nous donner une nouvelle clé permettant de coder les messages à partir d'une clé statique et d'une clé dépendant de paramètres temporels.
Par exemple, un message envoyé le 15 Avril 2021 sera codé via la clé `rb5CuefkDLyb9T2021-04-15`.

Le code a permis de déterminer l'utilisation de l'algorithme de chiffrement par bloc AES dans le mode ECB avec le schéma PKCS5Padding :

```
private byte[] save_msg(String string2) throws NoSuchAlgorithmException,
 ↪ NoSuchPaddingException, InvalidKeyException,
 ↪ InvalidParameterSpecException, IllegalBlockSizeException,
 ↪ BadPaddingException, IOException, JSONException {
        SecretKeySpec secretKeySpec = new
         ↪ SecretKeySpec(this.get_data().getBytes(), "AES");
        Cipher cipher = Cipher.getInstance((String)"AES/ECB/PKCS5Padding");
        cipher.init(1, (Key)secretKeySpec);
        return cipher.doFinal(string2.getBytes("UTF-8"));
}
```

Après avoir crypté le message de cette façon, on va rajouter `HHY` devant ce message pour que l'application puisse différencier les messages chiffrés de ceux qui ne le sont pas.

```
public String put(String string2) {
        try {
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append("HHY");

 ↪ stringBuilder.append(Base64.encodeToString((byte[])this.save_msg(string2),
 ↪ (int)2));
            return stringBuilder.toString();
        }
        catch (IOException | InvalidKeyException | NoSuchAlgorithmException |
         ↪ InvalidParameterSpecException | BadPaddingException |
         ↪ IllegalBlockSizeException | NoSuchPaddingException | JSONException
         ↪ noSuchPaddingException) {
            return "";
        }
}
```

Pour résumer, on va coder les messages de la manière suivante :

$$\text{HHY} + \text{base64}(\text{AES avec clé hybride}(\text{Message Clair}))$$

Pour décoder on va donc :

1. enlever `HHY` du message crypté
2. traduire depuis la base64
3. décoder AES grâce à la clé créée à partir de `rb5CuefkDLyb9T` et de la date d'envoi du message

Cela nous a permis de pouvoir pouvoir lire les 42 messages qui ont été jusqu'alors illisibles.

2.6.2 Messages décodés

Voici une partie de ces messages considérés comme les plus importants pour notre travail.

Heil Hydra 🐙**!** `6cKEMFlNU3NPs/bxhxPKQRyDXUrQK7a8tHtX5xBXdYQ=`

Conversation du 15 avril 2021 :

We need to meet you. Get to geneva asap, further instructions will follow.
HHYbm0jyH9MsbIM1DsyvNU9iDm63RQa9/1Fxfrs/A75m3wAM04VwDGphJ4m9IUTjPQ//3qbJmQk
xQQeu50WmK0zLHj93BiByuFEHsvHwzcfHsI

understood, was headed to geneva anyway, already on a train. S6+p9IajgCnkXIm0tou
uBzw/c5dSk4ZVzCEeA7BZmnR7+J5ybgmhLeXCyxpwovM8yFaK1NQtfAJXHKwHnMGIxg==

Good. Meet us at our base. Below the Turkish consulate. Delete this message once read!
HHYcdZD5nSpEbiNiPHaMn/eJR3qfnG1bBjxHLgN2Q91Jjx+DaAtq15BuatyPqSSH0ZOqLPbis2
11525r4RIkumQSF1p8xjb7tbOHzWN250Sqmoh9v+9kcE6lnWM0cYHo4ub

2.7 Skimming

Partie 3

Analysis

Conclusion

Suite à l'analyse du téléphone du suspect, nous avons été en mesure d'établir clairement :

- Qu'il fait partie de l'organisation fictive HYDRA